Thymeleaf介绍

Thymeleaf 是一个现代的 **服务器端 Java 模板引擎**，它被设计用来更好地处理 Web 和独立环境中的 XML、XHTML、HTML5 等模板。它的主要目标是提供一种优雅且高度可维护的模板创建方式。

Thymeleaf 最引人注目的特点之一是它的 **"自然模板" (Natural Templates)** 理念。这意味着 Thymeleaf 模板文件（通常是 `.html` 文件）可以在**浏览器中直接打开并正确显示**，即使它还没有被服务器处理。这是因为它使用 HTML 属性或标签作为其模板语法，而这些属性或标签在浏览器解析时会被忽略或以一种无害的方式呈现。

例如，一个 Thymeleaf 模板片段可能看起来像这样：

```
<p th:text="${user.name}">John Doe</p>
```

在浏览器中直接打开这个 HTML 文件时，你会看到 "John Doe"。但当 Thymeleaf 引擎处理它时，`th:text` 属性会指示引擎用 `${user.name}` 的实际值替换 "John Doe"。这种特性对于前端开发人员和设计师来说非常友好，因为他们可以在没有后端服务的情况下直接在浏览器中预览和设计页面。

以下是 Thymeleaf 的几个关键特性：

- **服务器端渲染：** Thymeleaf 在服务器端处理模板，将动态数据填充到 HTML 结构中，然后将最终的 HTML 发送给客户端浏览器。
- **HTML5 兼容：** 它完全支持 HTML5 规范，并且不会破坏 HTML 结构。
- **与 Spring Framework 深度集成：** Thymeleaf 是 Spring 官方推荐的模板引擎之一，与 Spring MVC、Spring Security 等模块有非常好的集成，提供了许多方便的特性，如表单绑定、消息国际化、URL 处理等。
- **多方言支持：** 除了 HTML 之外，Thymeleaf 也支持 XML、CSS、JavaScript 等其他文本格式作为模板。你可以根据需要使用不同的 "方言" (Dialects) 来处理这些文件。
- **模块化和可扩展性：** 它的设计是模块化的，允许开发者创建自定义的方言和处理逻辑来扩展其功能。
- **强大的表达式语言：** Thymeleaf 提供了一套丰富而强大的表达式语言，用于访问数据、进行条件判断、循环遍历等操作，例如：
    - **变量表达式：** `${user.name}`
    - **选择变量表达式：** `*{lastName}` (在 `th:object` 作用域内)
    - **消息表达式：** `#{message.code}` (用于国际化)
    - **链接表达式：** `@{/users/profile(id=${user.id})}` (用于构建 URL)
    - **片段表达式：** `~{fragments :: header}` (用于复用 HTML 片段)

Thymeleaf使用

表达式

关于Thymeleaf的使用主要是集中在它的几种表达式上

柔狗安全内部培训资料

1. 变量表达式 (Variable Expressions) `${...}`

作用： 用于访问模型（Model）中的变量。这是最常用的表达式类型，你可以用它来获取在 Spring Controller 中通过 `model.addAttribute()` 添加到上下文中的任何数据。它也可以用来访问 JavaBean 属性、Map 键值、数组元素等。

语法： `${object.property}`, `${map.key}`, `${array[index]}`

示例： 假设你在 Controller 中有：

```
// Spring Controller
@GetMapping("/example")
public String example(Model model) {
    User user = new User("Alice", "alice@example.com", "user");
    model.addAttribute("user", user);
    model.addAttribute("appName", "My Application");
    model.addAttribute("features", Arrays.asList("Login", "Register",
"Dashboard"));
    return "example";
}

// User.java (POJO)
public class User {
    private String name;
    private String email;
    private String role;
    // Getters and Setters
}
```

```
<p th:text="${appName}">默认应用名称</p>
<p>用户名: <span th:text="${user.name}"></span></p>
<p>邮箱: <span th:text="${user.email}"></span></p>
<p>角色: <span th:text="${user.role}"></span></p>
<p>第一个功能: <span th:text="${features[0]}"></span></p>
```

某公司全内部培训资料

2. 选择表达式 (Selection Expressions) `*{...}`

作用： 类似于变量表达式，但它作用于一个**预先选择的对象**。它通常与 `th:object` 属性结合使用，这样你就不必重复写对象的名称。这在表单处理时特别有用。

语法： `*{property}`

示例： 继续上面的 Controller 示例：

HTML

```
<div th:object="${user}">
    <p>用户名: <span th:text="*{name}"></span></p>
    <p>邮箱: <span th:text="*{email}"></span></p>
    <p>角色: <span th:text="*{role}"></span></p>
    </div>

<form th:action="@{/update}" th:object="${user}" method="post">
    <input type="text" th:field="*{name}" /> <input type="email" th:field="*
{email}" />
    <input type="submit" value="更新" />
</form>
```

3. 消息 (i18n) 表达式 (Message (i18n) Expressions) `#{...}`

作用： 用于国际化（i18n）文本。它根据当前语言环境从消息源（如 `.properties` 文件）中查找对应的文本。

语法： `#{message.code}`, `#{message.code(arg1, arg2)}`

示例： 假设你的 `messages_zh_CN.properties` 文件中有：

```
welcome.message=欢迎您, {0}!
page.title=用户仪表板
```

```
<head>
    <title th:text="#{page.title}">默认标题</title>
</head>
<body>
    <h1 th:text="#{welcome.message(${user.name})}"></h1>
</body>
```

4. 链接 (URL) 表达式 (Link (URL) Expressions) `@{...}`

作用： 用于构建 URL。Thymeleaf 会自动处理 URL 的上下文路径，并可以方便地添加参数。它能确保生成的 URL 是正确且可移植的。

语法：

- **绝对 URL：** `@{/path/to/resource}`
- **带参数的 URL：** `@{/path/to/resource(param1=${value1}, param2=${value2})}`
- **协议相对 URL：** `@{//host/path}`
- **服务器相对 URL：** `@{/path}`
- **上下文相对 URL：** `@{path}`
- **参数 URL：** `@{path?param1=value}` (不推荐，建议使用括号语法)

示例： 假设你的应用部署在 `/myapp` 路径下，用户 ID 是 123。

柔韵安全内部培训资料

```
<a th:href="@{/users/profile(id=${user.id})}">查看用户资料</a>

<img th:src="@{/images/logo.png}" />

<a th:href="@{/logout}">退出</a>
```

5. 片段表达式 (Fragment Expressions) `~{...}`

作用： 用于引用和插入模板片段。这对于布局（layout）和组件化非常有用，可以避免重复编写相同的 HTML 结构。

语法： `~{templatename :: fragmentname}` 或 `~{:: fragmentname}`（当前文件片段）

示例： 假设 `fragments.html` 文件中有：

```
<div th:fragment="header">
    <h1>这是一个头部</h1>
    <nav>...</nav>
</div>

<div th:fragment="footer">
    <p>&copy; 2025 All Rights Reserved.</p>
</div>
```

```
<body>
    <div th:replace="~{fragments :: header}"></div>

    <main>
        </main>

    <div th:insert="~{fragments :: footer}"></div>
</body>
```

- `th:replace`： 用片段的内容完全替换包含 `th:replace` 的标签。
- `th:insert`： 将片段的内容插入到包含 `th:insert` 的标签**内部**。
- `th:include`： 类似于 `th:insert`，但不会包含片段的根标签（Thymeleaf 3 推荐使用 `th:insert`）。

字面量

字面量是模板中直接写死的值。

1. 文本 (Text Literals)

用单引号 `'` 包裹的字符串。 **示例：** `'Hello World'`, `'This is a test!'`

HTML

```
<p th:text="'欢迎来到我的网站!'"></p>
```

柴狗安全内部培训资料

2. 数值 (Number Literals)

直接写入数字。 **示例：** `0`, `34`, `3.0`, `12.3`

```
<p th:text="${quantity * 2.5}">总价</p>
```

3. 布尔 (Boolean Literals)

```
true` 或 `false`。 **示例：** `true`, `false
```

```
<div th:if="${user.active == true}">用户是活跃的</div>
```

4. Null (Null Literal)

```
null`。 **示例：** `null
```

```
<p th:if="${user.address != null}" th:text="${user.address}"></p>
```

5. 字面标记 (Literal Tokens)

不带引号，作为文本或特殊含义使用的单词。 **示例：** `one`, `sometext`, `main`, `admin` 等。当这些词被用于表达式中且不被引号包裹时，Thymeleaf 会将其视为字符串字面量。

HTML

```
<p th:if="${user.role eq 'admin'}">您是管理员</p>
```

文本操作

1. 字符串拼接 `+`

用于连接字符串和其他表达式的值。

示例：

```
<p th:text="'用户ID: ' + ${user.id}">用户ID</p>
```

2. 文本替换 (Text Substitution) `|...|`

提供一种方便的方式来混合文本和表达式，无需使用 `+` 进行多次拼接。

语法： `|Text with ${expression} and another ${expression2}|`

示例：

柔和安全内部培训资料

HTML

```
<p th:text="|欢迎 ${user.name}, 您的邮箱是 ${user.email}|"></p>
```

算术操作

用于数值的加减乘除和取模。

1. 二元运算符: `+`, `-`, `*`, `/`, `%`

示例:

```
<p th:text="${price * quantity}">总价格</p>
<p th:text="${total / items}">平均价格</p>
```

2. 减号 (单目运算符): `-`

用于表示负数或取反。

示例:

```
<p th:text="${-temperature}">负温度</p>
```

布尔操作 (Boolean Operations)

用于逻辑判断。

1. 二元运算符: `and`, `or`

示例:

```
<div th:if="${user.isAdmin() and user.isActive()}">显示管理员面板</div>
<div th:if="${user.isGuest() or user.isTrial()}">欢迎访客或试用用户</div>
```

2. 布尔否定 (一元运算符): `!`, `not`

示例:

```
<div th:if="${not user.isAdmin()}">您不是管理员</div>
<div th:if="${!user.isActive()}">用户不活跃</div>
```

比较和等价

用于比较两个值。

柔狗安全内部培训资料

1. 比较: `>`, `<`, `>=`, `<=`, `gt`, `lt`, `ge`, `le`

Thymeleaf 提供了符号和文本两种形式。

- `gt`: Greater Than (大于)
- `lt`: Less Than (小于)
- `ge`: Greater than or Equal (大于等于)
- `le`: Less than or Equal (小于等于)

示例:

```
<p th:if="${item.stock gt 0}">有库存</p>
<p th:if="${age >= 18}">成年人</p>
```

2. 等价: `==`, `!=`, `eq`, `ne`

Thymeleaf 提供了符号和文本两种形式。

- `eq`: Equal (等于)
- `ne`: Not Equal (不等于)

示例:

```
<div th:if="${user.role eq 'admin'}">管理员专用内容</div>
<div th:if="${user.status != 'inactive'}">状态正常</div>
```

条件运算符

用于根据条件返回不同的值。

1. If-then: `(if) ? (then)`

如果条件为真，则返回 `then` 部分的值；否则返回 `null`。

示例:

```
<p th:text="${user.isPremium()} ? '高级用户'"></p>
```

2. If-then-else: `(if) ? (then) : (else)`

如果条件为真，则返回 `then` 部分的值；否则返回 `else` 部分的值。

示例:

```
<p th:text="${user.gender == 'M'} ? '男' : '女'"></p>
<span th:class="${user.isAdmin()} ? 'admin-badge' : 'user-badge'"></span>
```

柔狗安全内部培训资料

3. Default: `(value) ?: (defaultvalue)`

如果 `value` 为 `null` 或空，则返回 `defaultvalue`；否则返回 `value`。这是一个非常方便的空值处理操作符。

示例：

```
<p th:text="${user.nickname ?: '未设置昵称'}"></p>
<img th:src="${user.avatarUrl ?: '/images/default-avatar.png'}" />
```

特殊标记 (Special Token)

No-Operation (无操作): `_`

作用： 如前面详细介绍的，它表示一个“无操作”，不产生任何输出。它不是一个值（如 `null` 或空字符串），而是一个指示符，告诉 Thymeleaf 在此处不做任何处理。

示例： 场景一：在条件表达式中，当条件不满足时不渲染任何内容

HTML

```
<span th:text="${user.hasPermission('EDIT')} ? '允许编辑' : _"></span>
```

场景二：在动态属性中，当条件不满足时不添加属性

HTML

```
<input type="text" th:attr="placeholder=${user.isNew()} ? '请输入您的用户名' : _" />
```

与 `null` 和空字符串 `''` 的区别：

- `null`： 表示值的缺失。当它被用于 `th:text` 时，通常会渲染成空字符串。
- `''` **(空字符串)**： 表示一个长度为零的字符串。当它被用于 `th:text` 时，会渲染为空字符串。
- `_` **(无操作)**： 表示引擎在此处不应进行任何操作或生成任何输出。它更强调的是**行为**而非**值**。

在大多数情况下，`null` 和 `_` 的视觉效果可能是相似的（都导致没有文本输出），但 `_` 明确传达了“有意不渲染”的意图，并且在处理属性时，`_` 会导致属性被完全省略，而 `null` 或 `''` 可能会导致属性被渲染但值为 `null` 或空字符串（这取决于具体的 Thymeleaf 属性处理器）。

Thymeleaf 模板注入漏洞

介绍

Thymeleaf 服务器端模板注入 (Server-Side Template Injection, SSTI) 漏洞，通常发生在应用程序**不安全地使用用户提供的动态输入来构建或解析 Thymeleaf 模板时。**

简单来说，就是当你的应用程序将**用户可控的数据**直接作为 **Thymeleaf 模板的一部分**进行渲染时，攻击者可以通过**精心构造恶意的 Thymeleaf 表达式**作为输入发送给服务器。Thymeleaf 引擎在处理这些数据时，会将其误认为是合法的模板指令或表达式，并尝试在服务器端执行这些恶意代码。

美狗安全内部培训资料

攻击原理

Thymeleaf 强大的**表达式语言**是其核心特性之一，它允许开发者动态地设置模板中的值、执行逻辑、调用方法等。例如，`${user.name}` 会被替换为用户的名字。

然而，如果攻击者能够控制这个表达式的**内容**，他们就可以注入恶意的 Java 代码，让 Thymeleaf 引擎在服务器上执行。一个经典的例子就是利用 Java 的反射机制来执行系统命令：

```
${T(java.lang.Runtime).getRuntime().exec('calc')}
```

这个表达式的含义是：

- `T(...)`：Thymeleaf 表达式中的一个特殊语法，用于引用 Java 类。
- `java.lang.Runtime`：Java 标准库中用于与运行时环境交互的类。
- `.getRuntime()`：获取 `Runtime` 类的单例实例。
- `.exec('calc')`：调用 `Runtime` 实例的 `exec` 方法来执行一个系统命令。在这里，它会尝试启动计算器程序 (`calc` 命令在 Windows 上有效)。

如果应用程序没有对用户输入进行严格的验证和过滤，并且将包含此恶意表达式的字符串直接传递给 Thymeleaf 引擎进行解析，那么 `calc` 命令就会在运行应用程序的服务器上被执行。

漏洞影响

Thymeleaf SSTI 漏洞的直接后果是**服务器上的远程代码执行 (Remote Code Execution, RCE)**。这意味着攻击者可以：

- **完全控制服务器：** 执行任意系统命令，如上传后门、修改文件、删除数据等。
- **访问敏感数据：** 读取数据库凭据、用户数据、配置文件等任何服务器上的敏感信息。
- **成为跳板：** 利用受感染的服务器作为跳板，攻击内网中的其他系统。

这种漏洞的危害性极高，因为它可能导致整个系统的沦陷。

漏洞版本及修复

Thymeleaf 3.0.0 至 3.0.11 版本存在这种模板注入漏洞。这些版本在处理一些动态表达式时可能存在缺陷，允许未经授权的代码执行。

Thymeleaf 3.0.12 及以后版本已经对这些已知的模板注入漏洞进行了修复。修复主要集中在**更严格的输入校验和沙箱机制**，以防止恶意表达式的执行。它限制了可以在模板中直接访问的类和方法，尤其是那些可能导致代码执行的敏感类。

Bypass 方式的存在

尽管 Thymeleaf 官方对漏洞进行了修复，但就像许多安全补丁一样，**仍然存在一些绕过 (Bypass) 的方式**。这些绕过通常利用以下几种情况：

1. **不完全的沙箱限制：** 修复可能没有完全覆盖所有 Java 类和方法的反射访问路径。攻击者可能会发现新的、未被列入黑名单的类或方法来间接执行代码。
2. **不安全的应用程序代码：** 即使 Thymeleaf 引擎本身得到修复，如果应用程序代码仍然将用户输入直接拼接成敏感的模板逻辑（例如，直接构建 `SpringEL` 表达式并将其注入到 `th:text` 或其他属性中），那么新的注入点仍然可能出现。

柴狗安全内部培训资料

3. **自定义方言或处理器中的漏洞：** 如果开发者使用了自定义的 Thymeleaf 方言或表达式处理器，而这些自定义代码没有进行严格的输入验证，也可能引入新的注入点。
4. **环境差异：** 在特定的 Java 版本、Spring 版本或其他依赖环境下，可能出现未预料到的交互方式，从而绕过安全限制。

因此，即使升级到最新版本的 Thymeleaf，**开发者也绝不能放松警惕。**关键在于**永远不要相信用户输入。**任何来自用户的数据，在被用作模板内容、模板名称或任何可能被 Thymeleaf 引擎解析的部分之前，都必须经过**严格的验证、过滤或沙箱化处理。**

防御的核心思想是： 永远不要允许用户输入直接影响模板的**结构或表达式本身。**用户输入应该仅仅作为**数据**传递给模板，而不是作为**模板逻辑。**

真正的漏洞核心

Thymeleaf 底层会利用 Spring Framework 的表达式语言，即 Spring Expression Language (SpEL) 来解析和执行更复杂的表达式。

Thymeleaf 与 SpEL 的关系

1. **Thymeleaf 表达式是外壳：** Thymeleaf 有自己的标准表达式语法，比如 `${...}`、`*{...}` 等。这些是 Thymeleaf 模板引擎识别和处理的语法。
2. **SpEL 是核心解析器：** 当 Thymeleaf 遇到这些表达式时，如果需要执行复杂的逻辑（比如访问静态方法、调用构造函数、执行类方法、或者进行复杂的条件判断等），它会将表达式的**内部内容**委托给 **SpEL 引擎**进行解析和求值。
3. `T()` **的作用：** SpEL 提供了 `T()` 运算符，用于**引用 Java 类型 (Class)。**一旦引用了类型，你就可以访问该类型的静态方法、静态字段，甚至创建新的对象（尽管在模板中创建对象通常不推荐）。

SpEL 正是为了满足这些需求而设计的。Spring Framework 大量使用 SpEL，而 Thymeleaf 作为 Spring 官方推荐的模板引擎，自然也深度整合了 SpEL 来扩展其表达式能力。

POC

```
${T (java.lang.Runtime).getRuntime().exec("calc.exe")}
$%7bnew%20java.util.Scanner(T(java.lang.Runtime).getRuntime().exec("calc").getInp
utStream()).next()%7d
__$%7bnew%20java.util.Scanner(T(java.lang.Runtime).getRuntime().exec(%22id%22).ge
tInputStream()).next()%7d__::.x
__${}__
```

修改方案

1、升高版本

Thymeleaf 3.x 的最新版本是 **3.0.15**（截至2025年7月5日）

柔狗安全内部培训资料

2、使用@ResponseBody或@RestController修饰

@ResponseBody 是一个Spring框架中的注解，它用于指示该方法的返回值应该直接写入HTTP响应正文ResponseBody中，而不是通过视图解析器进行渲染。使用 @ResponseBody 注解可以将方法返回值以JSON、XML等格式直接写入HTTP响应体中，常用于返回 RESTful API 接口的响应数据。
@RestController是Spring框架中的一个注解，它结合了@Controller和 @ResponseBody注解的功能，用于简化 RESTful Web 服务开发

- `@ResponseBody` **的作用**：
  该注解告诉 Spring **直接将返回值作为 HTTP 响应体**，而不是视图名称（View Name）。因此：
  - **不会触发视图解析器**（如 Thymeleaf、JSP）。
  - **不会解析 `::` 片段表达式**，字符串 `"welcome :: " + section` 会原样返回给客户端。
  - **无模板注入风险**，即使 `section` 包含 `${...}` 等表达式，也会被当作普通文本。

3、使用redirect:或forward:修饰

根据springboot定义，如果名称以redirect:开头，则不再调用ThymeleafView解析，调用RedirectView去解析controller的返回值

- `redirect:` **前缀的作用**：
  Spring MVC 会将 `redirect:` 开头的返回值识别为重定向指令：
  - **直接跳转到指定 URL**，不经过视图解析器。
  - **不解析 Thymeleaf 表达式**，即使 `url` 包含 `${...}` 也会被视为普通字符串拼接。
  - **重定向目标由客户端处理**，服务端仅返回 `302` 响应和 `Location` 头。

4、设置为HttpServletResponse

由于controller的参数被设置为HttpServletResponse，Spring认为它已经处理了HTTP Response，因此不会发生视图名称解析，也就不会存在模板注入漏洞了

柔狗安全内部培训资料